

ARQUITETURA E DESENVOLVIMENTO EM JAVA

FASE 1 | AULA 02 -

CRIAÇÃO E CONFIGURAÇÃO DE UM PROJETO SPRING MVC

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
O QUE VOCÊ VIU NESTA AULA?	12
REFERÊNCIAS.....	13

EXEMPLO

O QUE VEM POR AÍ?

Nesta aula, criaremos um microserviço para a gestão de carros em uma locadora utilizando Spring Boot e Maven. O Maven é uma ferramenta de automação de compilação e gerenciamento de dependências.

Vamos começar configurando nosso projeto utilizando o Spring Initializer. Essa poderosa ferramenta nos permitirá gerar a estrutura de diretórios e configurações iniciais necessárias para um projeto Spring Boot de forma rápida e eficiente.

Em seguida, configuraremos as dependências do projeto utilizando o Maven. Definiremos todas as bibliotecas necessárias no arquivo pom.xml, garantindo que nosso projeto tenha todas as ferramentas essenciais para funcionar corretamente. O Maven nos auxilia a gerenciar essas dependências de maneira eficiente, automatizando o download e a integração de bibliotecas externas.

Após configurar as dependências, passaremos para a criação de controladores e o mapeamento de rotas utilizando as anotações do Spring Web MVC. Essa etapa é fundamental para definir como nossa aplicação responderá a requisições HTTP, permitindo a criação de uma API RESTful robusta e eficiente. Utilizaremos as anotações fornecidas pelo Spring para mapear as rotas e gerenciar as interações com o cliente.

HANDS ON

Vamos criar um microserviço para a gestão de carros de uma locadora utilizando Spring Boot e Maven. Começaremos configurando o projeto e a estrutura de diretórios com o Spring Initializer, selecionando as dependências essenciais, como Spring Web. Em seguida, configuraremos as dependências no arquivo pom.xml, garantindo que o Maven gerencie automaticamente todas as bibliotecas necessárias para o funcionamento do projeto.

Com a base do projeto configurada, avançaremos para a implementação dos controladores e mapeamento de rotas. Utilizaremos as anotações do Spring MVC para criar controladores RESTful que gerenciarão as requisições HTTP, permitindo operações de CRUD para a gestão de carros na locadora. Ao final da aula, teremos iniciado o desenvolvimento de uma API, proporcionando uma compreensão sólida de como construir e implantar APIs utilizando Spring Boot e Maven de maneira eficiente e escalável.

SAIBA MAIS

Vimos que o Spring foi construído em cima de injeção de dependências (DI) e inversão de controle (IOC), mas o que são esses paradigmas?

A Inversão de Controle (IoC) é um princípio de design que promove maior modularidade, testabilidade, reutilização e baixo acoplamento em sistemas de software. Diferentemente de abordagens tradicionais, como a programação procedural, onde o fluxo de controle é explícito no código, o fluxo da IoC é invertido, delegando a execução para bibliotecas ou frameworks externos.

Na prática, isso significa que, em vez de o código do aplicativo chamar diretamente métodos ou funções, ele define comportamentos e dependências e o framework ou container de IoC é responsável por gerenciar esses componentes e suas interações. Esse conceito é frequentemente associado ao Princípio de Hollywood ("Não nos chame, nós chamamos você"), destacando a ideia de que componentes são chamados quando necessários, aumentando a flexibilidade e a modularidade do código.

A IoC é frequentemente implementada em Java usando a Injeção de Dependência (DI), uma técnica que permite que objetos sejam passados como dependências em vez de serem instanciados diretamente dentro de uma classe. Por exemplo, considere um controlador que precisa acessar dados de clientes:

```
public class OrderController {
    private final CustomerRepository repository;

    public OrderController(CustomerRepository repository)
    {
        this.repository = repository;
    }

    @PostMapping("/v1/orders")
    public String placeOrder(@RequestBody
PlaceOrderCommand command) {
        Customer customer =
repository.findById(command.getCustomerId());
        // lógica adicional
        return "Order placed";
    }
}
```

```
}
```

Código-fonte 1 – Exemplo de código-fonte JAVA
Fonte: elaborado pelo autor (2024)

Nesse exemplo, o `OrderController` depende de `CustomerRepository` para obter dados de clientes. Essa dependência é injetada através do construtor, promovendo um design mais flexível e testável. Para configurar a IoC e DI em um aplicativo Spring, você definiria `CustomerRepository` como um bean:

```
@Configuration
public class AppConfig {
    @Bean
    public CustomerRepository customerRepository() {
        return new CustomerRepository();
    }

    @Bean
    public OrderController
orderController(CustomerRepository repository) {
        return new OrderController(repository);
    }
}
```

Código-fonte 2 – Exemplo de código-fonte JAVA (2)
Fonte: elaborado pelo autor (2024)

Com essa configuração, o Spring Framework gerencia a criação e a injeção de dependências, permitindo um baixo acoplamento e facilitando a manutenção e a escalabilidade do aplicativo. Além disso, ao usar interfaces, como no Princípio da Inversão de Dependência (DIP), você pode aumentar ainda mais a flexibilidade:

```
public interface ICustomerRepository {
    Customer findById(UUID id);
}

public class CustomerRepository implements
ICustomerRepository {
    // Implementação dos métodos
}

@Configuration
public class AppConfig {
    @Bean
    public ICustomerRepository customerRepository() {
        return new CustomerRepository();
    }
}
```

```
    }

    @Bean
    public OrderController
orderController(ICustomerRepository repository) {
        return new OrderController(repository);
    }
}
```

Código-fonte 3 – Exemplo de código-fonte JAVA (3)
Fonte: elaborado pelo autor (2024)

Neste cenário, o controlador depende de uma abstração (ICustomerRepository), permitindo a troca fácil de implementações, o que é útil para testes e manutenção. Assim, IoC, DI e DIP trabalham juntos para criar um sistema mais modular, testável e de baixo acoplamento.

Outro ponto abordado é o gerenciamento de dependências. As dependências referem-se a bibliotecas externas, frameworks e módulos necessários para que uma aplicação funcione corretamente. Gerenciar essas dependências é uma tarefa importante, pois envolve a inclusão, atualização e resolução de conflitos entre diferentes versões de bibliotecas.

No ecossistema Java, o gerenciamento de dependências é essencial para simplificar o processo de desenvolvimento. Antes de surgirem as ferramentas de gerenciamento, as pessoas desenvolvedoras precisavam baixar manualmente as bibliotecas necessárias, configurar o classpath e lidar com problemas de compatibilidade entre diferentes versões.

O Apache Maven foi uma das primeiras ferramentas populares de gerenciamento de dependências no mundo Java, surgindo em 2002. Ele introduziu um modelo de projeto baseado em arquivos de configuração (pom.xml), que descrevem as dependências, plugins e outras informações necessárias para o build do projeto. O Maven simplificou o processo de download e gerenciamento de dependências, além de fornecer funcionalidades para compilação, teste e empacotamento de projetos.

O Apache Maven, que é uma ferramenta de automação de compilação e gerenciamento de dependências amplamente utilizada no ecossistema Java. Ele

ajuda pessoas desenvolvedoras a simplificar o processo de construção, teste e empacotamento de projetos, além de facilitar a gestão das dependências.

O arquivo "pom.xml" (Project Object Model) é o arquivo de configuração principal do Maven. Ele descreve as informações do projeto, como seu nome, versão, descrição e os metadados necessários para o Maven gerenciar as dependências, plugins e tarefas de build.

O pom.xml contém seções específicas onde você pode definir as dependências do projeto, incluindo as bibliotecas necessárias para o seu funcionamento. O Maven utiliza essas informações para resolver as dependências automaticamente, baixando as versões corretas das bibliotecas de repositórios online (como o Maven Central) e configurando-as no projeto.

O pom.xml também permite a configuração de plugins, que são extensões do Maven que adicionam funcionalidades extras ao processo de build. Por exemplo, você pode usar plugins para executar testes automatizados, gerar documentação, empacotar o projeto em um arquivo JAR ou WAR, entre outras tarefas.

Uma das maiores vantagens do Maven e do pom.xml é a facilidade de uso e a padronização. Com o arquivo de configuração, é possível compartilhar e replicar facilmente as dependências e configurações do projeto em diferentes ambientes. Além disso, o Maven lida automaticamente com a resolução de dependências transitivas, ou seja, quando uma biblioteca depende de outras bibliotecas, o Maven cuida de baixar todas as dependências necessárias.

Outra vantagem é a integração com IDEs (Ambientes de Desenvolvimento Integrados) como o Eclipse e o IntelliJ IDEA. As IDEs são capazes de importar projetos Maven, reconhecendo automaticamente as dependências e oferecendo recursos como autocompletar e navegação entre classes. Outro gerenciador de pacotes é o Gradle, uma ferramenta de automação de compilação e gerenciamento de dependências que oferece uma abordagem mais flexível e poderosa em comparação ao Apache Maven. O Gradle utiliza uma DSL (Domain-Specific Language) baseada em Groovy ou Kotlin para definir as tarefas de build e configurações do projeto.

Groovy é uma linguagem de programação dinâmica e orientada a objetos que roda na JVM (Java Virtual Machine). Ela combina elementos do Java e de outras linguagens, como Python e Ruby, oferecendo uma sintaxe mais concisa e expressiva.

No contexto do Gradle, o Groovy é usado para escrever os scripts de build, tornando a configuração do projeto mais legível e fácil de entender.

Uma das principais vantagens do Gradle é sua flexibilidade. Ele permite que pessoas desenvolvedoras definam tarefas personalizadas e automatizem praticamente qualquer aspecto do processo de construção de um projeto. Além disso, o Gradle suporta a modularização do build, o que significa que é possível dividir o projeto em diferentes módulos e configurar builds independentes para cada um deles.

O uso do Groovy no Gradle também traz benefícios para pessoas desenvolvedoras. A sintaxe concisa e a possibilidade de utilizar construções como closures e DSLs facilitam a configuração e personalização do build. O Groovy também oferece recursos poderosos, como metaprogramação, que permitem estender a funcionalidade do Gradle de forma flexível.

Também é uma vantagem do Gradle a integração com o ecossistema do Java e a facilidade de uso com IDEs, como o IntelliJ IDEA e o Eclipse. O Gradle pode importar projetos existentes criados com o Maven, permitindo a migração gradual de projetos para o Gradle. Além disso, ele suporta repositórios Maven, o que significa que é possível utilizar as mesmas dependências disponíveis no Maven Central e em outros repositórios Maven.

Tanto o Maven quanto o Gradle possuem repositórios online (como o Maven Central) nos quais as bibliotecas e suas versões estão disponíveis para download automático. Essas ferramentas resolvem automaticamente as dependências, garantindo que as versões corretas sejam baixadas e configuradas no projeto. Isso facilita o desenvolvimento, pois não é necessário se preocupar manualmente com a obtenção das dependências corretas e suas transitive dependencies.

O Maven e o Gradle são duas ferramentas populares de automação de compilação e gerenciamento de dependências no ecossistema Java. Embora ambos tenham funcionalidades semelhantes, existem algumas diferenças-chave entre eles:

- **Linguagem de configuração:** o Maven utiliza o formato XML para configurar o projeto por meio do arquivo pom.xml. Já o Gradle utiliza uma DSL (Domain-Specific Language) baseada em Groovy ou Kotlin, que oferece uma sintaxe mais concisa e expressiva. A sintaxe em DSL do Gradle é considerada mais legível e fácil de entender.

- **Flexibilidade:** o Gradle é conhecido por sua flexibilidade e poder de personalização. Ele permite definir tarefas de build personalizadas e automatizar praticamente qualquer aspecto do processo de construção de um projeto. O Maven, por outro lado, possui um conjunto fixo de plugins e convenções pré-definidas, o que pode limitar a flexibilidade em alguns casos.
- **Desempenho:** o Gradle é conhecido por ser mais rápido em comparação com o Maven, especialmente em projetos grandes ou complexos. O Gradle possui um mecanismo de execução incremental que identifica quais partes do build precisam ser reconstruídas, evitando compilações desnecessárias e melhorando o tempo de construção.
- **Migração:** o Gradle oferece uma experiência de migração mais suave para projetos que já utilizam o Maven. Ele pode importar projetos Maven existentes e converter automaticamente o arquivo pom.xml em um script de build do Gradle, permitindo a transição gradual para o Gradle.

O Gradle foi construído para suprir a necessidade de uma ferramenta de construção mais flexível e poderosa em comparação com o Maven. Ele foi projetado para lidar com projetos complexos, modularização, personalização e extensibilidade. O Gradle oferece a pessoas desenvolvedoras maior controle e capacidade de adaptação ao processo de construção de software, permitindo que personalizem e automatizem o build de acordo com as necessidades específicas de cada projeto.

O Gradle com Groovy permite escrever scripts de build concisos e expressivos, aproveitando a sintaxe da linguagem Groovy. Essa combinação oferece maior flexibilidade, poder de personalização e capacidade de automação, permitindo a criação de tarefas de build personalizadas e adaptadas às necessidades específicas do projeto.

Já o Gradle com Kotlin oferece uma alternativa à linguagem Groovy, utilizando a sintaxe mais moderna e concisa do Kotlin. O Gradle com Kotlin mantém as vantagens em termos de flexibilidade, personalização e automação do Gradle com Groovy, enquanto aproveita os recursos adicionais fornecidos pelo Kotlin, como segurança de tipo estático e suporte para programação funcional.

Em resumo, o Maven é baseado em XML e oferece um conjunto fixo de plugins e convenções pré-definidas. O Gradle, por sua vez, oferece uma linguagem de configuração flexível e poderosa, permitindo a personalização e automação do processo de construção. O Gradle com Groovy e o Gradle com Kotlin ampliam ainda mais as capacidades do Gradle, oferecendo uma sintaxe mais legível e expressiva para a configuração do projeto.



O QUE VOCÊ VIU NESTA AULA?

Nesta aula, exploramos o uso do Spring Initializer, Maven e a criação de controladores RESTful no desenvolvimento de um microserviço para a gestão de carros em uma locadora.

Primeiro, aprendemos a utilizar o Spring Initializer para configurar nosso projeto. Essa ferramenta nos permitiu gerar a estrutura inicial do projeto, escolhendo as dependências essenciais como Spring Web e Spring Data JPA, o que facilitou o início do desenvolvimento de maneira organizada e eficiente.

Em seguida, abordamos a configuração das dependências utilizando o Maven. Com o arquivo pom.xml, configuramos todas as bibliotecas necessárias para o projeto, permitindo que o Maven gerencie automaticamente o download e a integração dessas dependências, simplificando o processo de construção e manutenção do projeto.

Por fim, implementamos controladores RESTful utilizando as anotações do Spring MVC. Criamos controladores que gerenciam as requisições HTTP para operações de CRUD relacionadas à gestão de carros na locadora. Utilizamos anotações como `@RestController`, `@RequestMapping`, `@GetMapping` e `@PostMapping` para mapear rotas e definir métodos que processam essas requisições.

REFERÊNCIAS

BAELDUNG. **Spring Boot Gradle Plugin**. 2024. Disponível em: <<https://www.baeldung.com/spring-boot-gradle-plugin>>. Acesso em: 02 ago. 2024.

BALTA.IO. **Inversion of Control**. 2024. Disponível em: <<https://balta.io/blog/inversion-of-control>>. Acesso em: 02 ago. 2024.

GRADLE. **Building Spring Boot Web Applications Sample**. 2024. Disponível em: <https://docs.gradle.org/current/samples/sample_building_spring_boot_web_applications.html>. Acesso em: 02 ago. 2024.

PALAVRAS-CHAVE

Palavras-chave: API. DD. SPRING BOOT. MVC.

EXEMPLO



POSTECH